

Relatório Técnico - Projeto SISOP2: Replicação e Eleição de Líder

Daniel Gutschwager e Pedro Mews

¹Curso de Ciência da Computação
Universidade Federal do Rio Grande do Sul (UFRGS)
Porto Alegre, RS – Brasil

1. Descrição do Ambiente de Testes

Para o desenvolvimento, validação e testes de carga do sistema distribuído, utilizou-se um ambiente virtualizado baseado em contêineres Docker, garantindo a reprodutibilidade dos cenários de falha e o isolamento dos processos.

- **Sistema Operacional Hospedeiro:** macOS.
- **Ambiente de Execução:** Docker Desktop.
- **Compilador e Ferramentas:**
 - g++ (GCC) e Make.
 - CMake para automação da compilação do projeto.
- **Configuração de Hardware:** A execução foi realizada em máquina com processador de arquitetura moderna (Apple Silicon/Intel Core) e 16GB de memória RAM, recursos suficientes para suportar a execução simultânea de múltiplos contêineres (3 servidores e 2 clientes) sem gargalos de desempenho significativos.
- **Rede:** Foi utilizada uma rede virtual *bridge* configurada pelo Docker Compose (*sisop-net*), permitindo a comunicação via IP entre os contêineres na sub-rede 172.25.0.0/16.

2. Algoritmo de Eleição de Líder

Para garantir a disponibilidade do serviço na ausência do servidor principal, foi implementado um mecanismo de eleição para escolher um novo coordenador.

2.1. Justificativa da Escolha

É um algoritmo que prioriza o processo com o maior identificador (ID) para ser o líder, o que simplifica a lógica de decisão e recuperação: se o processo de maior ID está ativo, ele deve ser o líder.

2.2. Funcionamento Implementado

A implementação segue a lógica de troca de mensagens UDP:

1. **Monitoramento:** Os servidores secundários monitoram o Líder através de mensagens periódicas de *heartbeat*. Se um *heartbeat* não for recebido dentro de um tempo limite (configurado para 3 segundos), suspeita-se de falha.
2. **Eleição:** O servidor que detecta a falha envia uma mensagem de *ELECTION* para todos os pares com ID maior que o seu.
3. **Respostas:**

- Se um servidor recebe `ELECTION` de um ID menor, ele responde com `ACK` e inicia sua própria eleição.
 - Se o servidor não recebe respostas (timeout), ele se considera o vencedor.
4. **Coordenação:** O vencedor envia uma mensagem `COORDINATOR` a todos, assumindo a liderança.
 5. **Recuperação:** Se um servidor com ID maior se recupera e entra na rede, ele inicia uma eleição imediatamente, retomando a liderança (comportamento "valentão").

3. Replicação Passiva

A tolerância a falhas foi assegurada através do modelo de **Replicação Passiva** (Primary-Backup), garantindo que o estado do sistema (saldos e histórico) sobreviva à queda do líder.

3.1. Implementação e Desafios

No modelo implementado, apenas o Líder (Primário) processa as requisições de escrita dos clientes.

- **Fluxo:** Ao receber uma requisição, o Líder valida a operação. Se bem-sucedida, ele atualiza seu estado local e envia mensagens de `STATE_UPDATE` para todos os backups conhecidos antes de confirmar ao cliente.
- **Sincronização de Estado (Snapshots):** Um dos desafios encontrados foi a entrada de novos servidores ou a recuperação de servidores antigos que possuíam estado obsoleto. Para resolver isso, implementou-se um mecanismo de *Snapshot*. Quando um servidor secundário detecta um novo líder, ele solicita o estado atual (`SNAPSHOT_REQ`). O líder responde enviando o saldo e o último ID de requisição de todos os clientes registrados, além das estatísticas globais do banco (total transferido e número de transações).

4. Dificuldades Encontradas

Durante o desenvolvimento da segunda etapa, a equipe enfrentou alguns obstáculos técnicos significativos:

- **Conhecimento Prévio da Topologia (Discovery):** Uma das maiores dificuldades foi fazer o projeto funcionar de forma totalmente dinâmica sem o conhecimento prévio de todos os servidores. A tentativa de implementar uma descoberta dinâmica de servidores (onde servidores descobrem uns aos outros via broadcast ao iniciar) introduziu complexidade e instabilidade no sistema de eleição. **Portanto, ao inicializar um servidor, foi necessário informar explicitamente via linha de comando os IDs, IPs e portas de todos os outros servidores pares.** Essa configuração estática garantiu que o algoritmo de eleição funcionasse corretamente, pois cada nó sabia exatamente quem eram seus superiores hierárquicos.
- **Consistência de Estatísticas Globais:** Manter o "Saldo Total do Banco" consistente entre réplicas foi desafiador. Inicialmente, ao transferir o estado para um novo líder, apenas os saldos individuais eram copiados, fazendo com que as estatísticas globais fossem resetadas. Foi necessário criar tipos de pacotes específicos (`SNAPSHOT_STATS`) para sincronizar também os contadores globais.

5. Divisão de Tarefas

As responsabilidades de implementação foram divididas da seguinte maneira:

- **Pedro Oliveira Mews:** Focou na implementação da lógica de **Replicação Passiva**. Desenvolveu a classe `ReplicaManager`, a estrutura de dados `ServerState`, a lógica de aplicação de atualizações nos backups e o mecanismo de transferência de estado via Snapshots.
- **Daniel Gutschwager:** Focou na implementação do **Algoritmo de Eleição** e na infraestrutura. Desenvolveu a classe `Election`, o sistema de detecção de falhas (heartbeats), a configuração do ambiente Docker Compose e os scripts para testes de carga e injeção de falhas.